Nutrition Assistant

Full Stack Development (MERN Stack) Project Documentation

TEAM LEADER	**Naraharisetti Sai Nitchal**
TEAM MEMBERS	**Sundarapalli Siva Sankar, Edward Jones Devathoti, Nunna Tarun Sri Venkata Sai, SAIRAJU ADDAGARLA**
TECHNOLOGY STACK	**MongoDB · Express.js · React.js (Vite) · Node.js**
PROJECT DOMAIN	**Web Application — Full Stack Development**
DEPLOYMENT	**Static client hosting + persistent Node.js API + MongoDB Atlas**

MERN Stack · JWT Authentication · AI Nutrition Chatbot · RESTful APIs

Table of Contents

1. Introduction 3

2. Project Overview 4

3. System Architecture 5

4. Setup Instructions 7

5. Folder Structure 8

6. Running the Application 9

7. API Documentation 10

8. Authentication & Authorization 12

9. User Interface — Screenshots 13

10. Testing 19

11. Screenshots / Demo Walkthrough 20

12. Known Issues 21

13. Future Enhancements 22

1. Introduction

Project Title

Nutrition Assistant

Team Members & Roles

Name	Role
Naraharisetti Sai Nitchal	Team Leader — Full Stack Development, Architecture & Integration
Sundarapalli Siva Sankar	Team Member — Backend Development & Testing
Edward Jones Devathoti	Team Member — Frontend Development
Nunna Tarun Sri Venkata Sai	Team Member — Database & API Design
SAIRAJU ADDAGARLA	Team Member — UI/UX & AI Integration

Project Domain & Type

- Domain: Web Application
- Type: Full Stack Web Application
- Technology: MERN Stack (MongoDB, Express.js, React.js, Node.js)

Nutrition Assistant is a premium, full-stack web application built using the MERN Stack for comprehensive lifestyle, diet and water tracking. It gives users exact physical calculators, structured meal planners, database-backed food/recipe search, and an AI Nutrition Chatbot — all tied to a single personal profile. The platform digitizes what is normally spread across several disconnected tools: calorie/BMI calculators, food diaries, recipe books and nutrition advice.

2. Project Overview

Purpose

Nutrition Assistant aims to give everyday users a single place to understand their body metrics, track their daily food and water intake, discover suitable recipes, and get instant, personalized nutrition guidance — replacing the fragmented mix of calculators, spreadsheets and generic diet advice most people rely on today.

Problem Statement

Traditional nutrition tracking is scattered across separate calculators, food-logging apps and generic online advice, with no shared user profile connecting them. This makes it easy for users to lose track of their goals and give up on healthy habits. Nutrition Assistant addresses this gap with one integrated platform.

Key Features

- Secure JWT-based dual-token authentication (access + refresh) with bcryptjs password hashing
- BMI Calculator (metric/imperial) with category-based guidelines
- Calorie Calculator (BMR via Harris-Benedict formula + TDEE via activity multiplier) with goal-based targets
- Daily food & water tracker with automatic macro/hydration total recalculation
- Dashboard with weekly trend charts, BMI summary and dynamic, goal-based recommendations
- Recipe Explorer with search, filters, pagination and favorites
- Meal Planner for scheduling breakfast/lunch/dinner/snacks
- AI Nutrition Chatbot powered by Google Gemini, with an intelligent rule-based fallback
- Admin Dashboard for platform statistics, user management and system food curation

- Responsive, glassmorphic design with dark mode, inspired by Apple Health and Google Fit
- Cloudinary-backed image uploads (profile photos, recipe/meal images) with local-disk fallback

Core Modules

Module	Functionality
Authentication	Registration, login, JWT access/refresh tokens, bcrypt hashing, forgot/reset password
Profile Management	Demographics, goal, activity level, diet preference, medical conditions, allergies, photo
Calculators	BMI Calculator, Calorie (BMR/TDEE) Calculator, apply goal to profile
Tracker	Food logging per meal slot, water logging, auto-computed daily totals
Recipes & Meals	Recipe search/filter/favorites, custom recipe creation, meal planner
AI Assistant	Conversational nutrition guidance via Gemini API with fallback responses
Settings	Dark mode, unit system (metric/imperial), reminder interval
Admin	Platform stats, user management with cascade delete, system food curation

3. System Architecture

Nutrition Assistant follows a three-tier architecture that cleanly separates presentation, application, and data-persistence concerns, enabling scalability and maintainability.

3.1 Frontend (Presentation Layer)

The client is built with React 19 and Vite for a fast development and build pipeline. React Router DOM handles client-side routing across public pages and protected/admin routes. Tailwind CSS v4 combined with Framer Motion (animations), Chart.js/react-chartjs-2 (charts) and Lucide React (icons) deliver a responsive, glassmorphic interface.

- React.js 19 + Vite for componentized, fast-refresh UI development
- React Router DOM for protected and admin-only route handling
- Tailwind CSS v4, Framer Motion and Chart.js for styling, animation and data visualization
- Axios-based API layer with request/response interceptors for automatic token refresh

3.2 Backend (Application Layer)

Node.js and Express.js form the application layer, exposing RESTful APIs under /api. Helmet, CORS and express-rate-limit protect the API at the edge; JWT verification and Zod-based request validation run as middleware ahead of MVC-layered controllers (auth, tracker, recipe, meal, admin).

3.3 Database (Data Layer)

MongoDB, accessed through Mongoose ODM, stores all persistent data. Mongoose pre-save hooks automatically recalculate aggregate fields — such as a NutritionLog's totalNutrition and a WaterLog's amount — whenever their embedded sub-documents change.

Database Collections

Model	Purpose
User	Account credentials, demographics, goal, activity level, allergies, role
Settings	Dark mode, notifications, reminder interval, unit system — one per user
Food	Nutrition facts per 100g/serving; system foods (creator: null) or user-created
Recipe	Title, description, ingredients, instructions, cooking time, nutrition facts, image
Meal	User-planned meal templates by meal time (breakfast/lunch/dinner/snack)
NutritionLog	Per-date logged foods with serving counts and auto-computed daily totals
WaterLog	Per-date logged water entries with auto-computed daily total and goal

Model	Purpose
Favorite	Join collection linking a user to a favorited recipe (unique per pair)

High-Level Data Flow

Client (React + Vite) --HTTPS / REST (JSON)--> Server (Node.js + Express: Helmet/CORS/Rate-Limit → JWT Auth Middleware → Zod Validation → Controllers) --Mongoose ODM--> MongoDB Atlas (Users, Foods, Recipes, Meals, NutritionLogs, WaterLogs, Favorites, Settings). The Controller layer additionally calls the Google Gemini API for AI chat responses and Cloudinary for image uploads (with local disk as a fallback).

4. Setup Instructions

Prerequisites

- Node.js (v18 or higher)
- npm (bundled with Node.js)
- MongoDB (Atlas account or local instance on port 27017)
- Git for version control
- Visual Studio Code (recommended IDE)

Installation Steps

1. Clone the repository: git clone <repository-url> && cd nutrition-assistant

2. Install all dependencies from the root workspace: npm run install-all (this runs npm install --prefix server and npm install --prefix client --legacy-peer-deps)

Environment Variables

Create a server/.env file with the following keys:

PORT=5000 MONGODB_URI=your_mongodb_uri_here JWT_SECRET=super_secret_jwt_access_token_key JWT_REFRESH_SECRET=super_secret_jwt_refresh_token_key JWT_EXPIRE=1d JWT_REFRESH_EXPIRE=7d GEMINI_API_KEY=your_gemini_api_key_here

Optional Cloudinary keys (CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET) enable cloud image storage; without them, uploads fall back to local disk storage automatically.

Automatic Database Seeding

Running npm run seed wipes existing data and populates the database with demo accounts, standard macro foods, and healthy recipes.

Role	Email	Password
Admin	admin@assistant.com	admin123
Regular User	user@assistant.com	user123

5. Folder Structure

Client (React Frontend)

client/ |-- public/ |-- src/ | |-- assets/ | |-- components/ (ProtectedRoute) | |-- contexts/ (Auth, Theme, Notification) | |-- layouts/ (Navbar, Sidebar, Footer, AppLayout) | |-- pages/ (22 core pages) | |-- services/ (Axios API client) | |-- App.jsx | |-- main.jsx |-- index.html |-- vite.config.js |-- package.json

Server (Node.js Backend)

server/ |-- config/ (MongoDB & Cloudinary config) |-- controllers/ (Auth, Tracker, Recipe, Meal, Admin) |-- middleware/ (auth, validate, upload, error) |-- models/ (Mongoose schemas) |-- routes/ (Express route definitions) |-- services/ (Gemini AI service) |-- utils/ (seed.js) |-- validators/ (Zod schemas) |-- index.js |-- package.json

Testing

tests/ |-- calculations.test.js (BMI/BMR/TDEE unit tests) |-- api.test.js (API route integration tests)

6. Running the Application

Start the Backend Server

npm run dev-server # Server runs at http://localhost:5000

Start the Frontend (Client)

npm run dev-client # App runs at http://localhost:5173 (Vite dev server)

Once both servers are running, open the client URL in a browser and sign in using the seeded demo credentials from Section 4.

Deployment

The React client is deployed as a static site, while the Express server runs as a persistent Node.js service, connected to a managed MongoDB Atlas cluster.

7. API Documentation

All endpoints are prefixed with /api and return JSON responses. Protected routes require an Authorization: Bearer <token> header obtained after login.

7.1 Authentication Routes (/auth)

Method	Endpoint	Description
POST	/auth/register	Create user profile and settings; returns JWT
POST	/auth/login	Validate credentials; returns JWT
POST	/auth/refresh	Rotate refresh token and issue a new access token
POST	/auth/logout	Clear refresh token (Private)
GET	/auth/me	Get profile statistics (Private)
PUT	/auth/profile	Update demographics & upload avatar photo (Private)
POST	/auth/forgotpassword	Request password reset token

Method	Endpoint	Description
POST	/auth/resetpassword/:token	Update password

7.2 Hydration & Food Tracker Routes (/tracker)

Method	Endpoint	Description
GET	/tracker/dashboard	Daily summaries, weekly trends, recommendations (Private)
POST	/tracker/water	Log water consumption (Private)
GET	/tracker/water/:date?	Read water log entries for a date (Private)
DELETE	/tracker/water/entry/:logId/:entryId	Delete a logged water entry (Private)
POST	/tracker/food	Log food items under a meal slot (Private)
GET	/tracker/food/:date?	Fetch logged food details (Private)
DELETE	/tracker/food/:logId/:mealId	Delete a logged food item (Private)

7.3 Food, Recipe & Meal Routes (/foods, /recipes, /meals)

Method	Endpoint	Description
GET	/foods?search=	Search database food items (Private)
POST	/foods	Submit a user custom food item (Private)
GET	/recipes	Paginated recipe list with search & filters (Public)
GET	/recipes/:id	Recipe details (Public)
POST	/recipes	Add a custom or system recipe (Private)
POST	/recipes/favorite/:id	Bookmark/favorite toggle (Private)
GET	/recipes/my/favorites	Retrieve favorited recipes (Private)
GET	/meals	Get planned meal templates (Private)
POST	/meals	Save a planned meal template (Private)
DELETE	/meals/:id	Delete a planned meal preset (Private)

7.4 AI, Settings & Admin Routes

Method	Endpoint	Description
POST	/ai/chat	Query the AI Nutrition Chatbot (Private)
GET	/settings	Fetch settings object (Private)
PUT	/settings	Modify dark mode / unit system preferences (Private)
GET	/admin/stats	Platform analytics (Private/Admin)
GET	/admin/users	Registered users list (Private/Admin)
DELETE	/admin/users/:id	Cascade delete a user and associated data (Private/Admin)
POST	/admin/foods	Curate a standard system food (Private/Admin)

Example Response

GET /api/recipes/64f1c2... { "success": true, "data": { "title": "Protein-Packed Oatmeal Berry Bowl", "cookingTime": 10, "nutritionFacts": { "calories": 420, "protein": 30, "carbs": 45, "fat": 12, "fiber": 8 } } }

8. Authentication & Authorization

Nutrition Assistant implements stateless authentication using JSON Web Tokens (JWT) with dual access/refresh tokens, combined with role-based access control (RBAC) to separate regular user and administrator capabilities.

How it Works

- Passwords are hashed using bcryptjs before being stored in MongoDB — plaintext passwords are never persisted

- On successful login, the server issues a short-lived JWT access token and a longer-lived refresh token
- The refresh token is stored on the User document and rotated on every /auth/refresh call
- The client stores both tokens and attaches the access token as a Bearer token on subsequent API requests
- An Axios response interceptor automatically calls /auth/refresh and retries the original request on a 401
- Express middleware (protect) verifies the token signature and expiry on every protected route
- An admin middleware checks the decoded role against admin-only routes

Token Lifecycle

1. POST /api/auth/login -> { email, password } 2. Server verifies bcrypt hash, signs access token (short expiry) + refresh token (7 day expiry) 3. Client stores both tokens, redirects to dashboard 4. Every request: Authorization: Bearer <accessToken> 5. protect middleware verifies token -> req.user 6. On 401: client calls /auth/refresh with the stored refresh token and retries

Role-Based Access

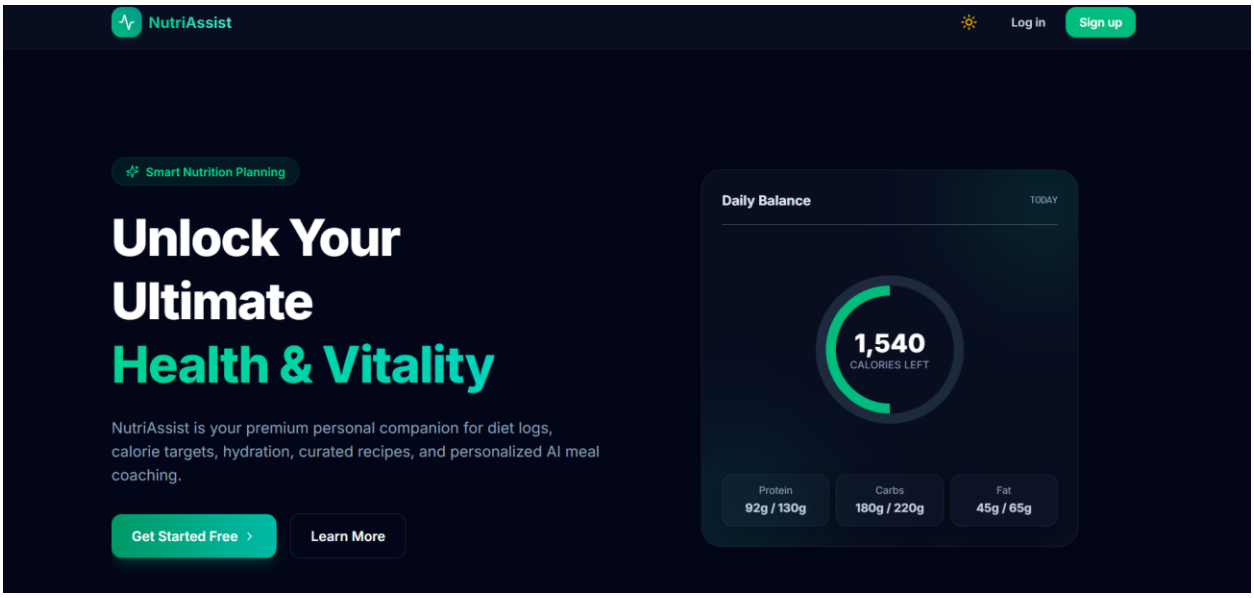
Role	Access
User	Profile, calculators, tracker, recipes, meals, AI chat, settings
Administrator	All user capabilities plus platform stats, user management, system food curation

9. User Interface — Screenshots

The following sections describe the primary interfaces of the Nutrition Assistant platform. Insert your own application screenshots at each indicated figure when preparing the final submission.

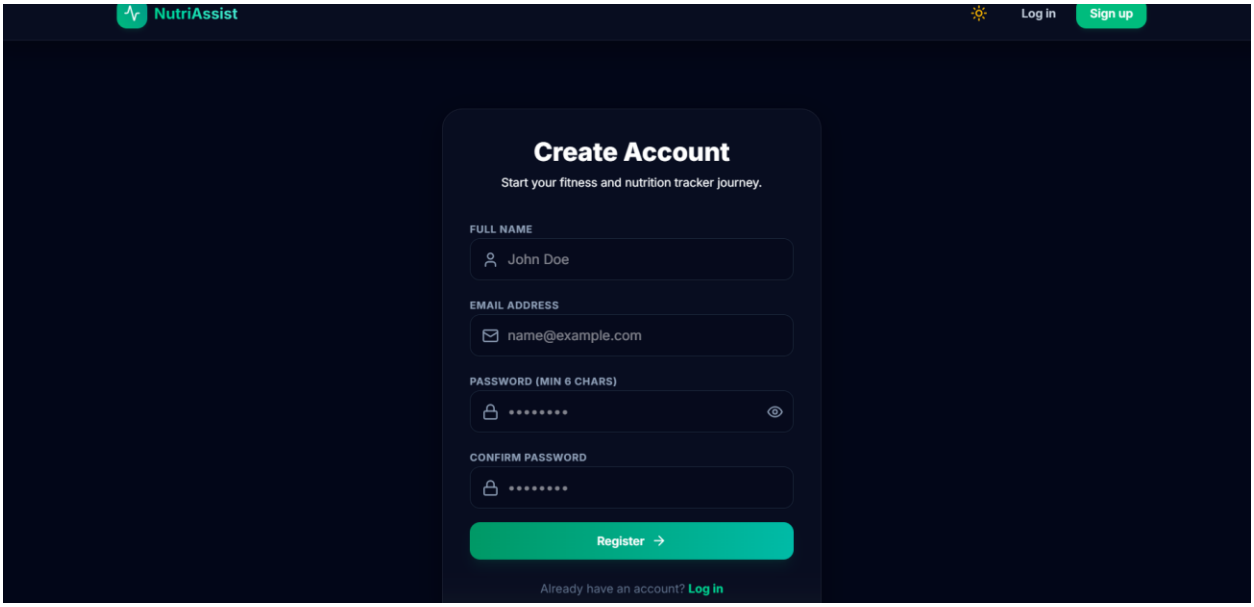
9.1 Landing Page

The public homepage introduces Nutrition Assistant's value proposition — calculators, tracking and the AI chatbot — with a glassmorphic hero section.



9.2 Registration & Login

A clean sign-up/sign-in flow collects name, email and password, issuing JWT tokens on success.

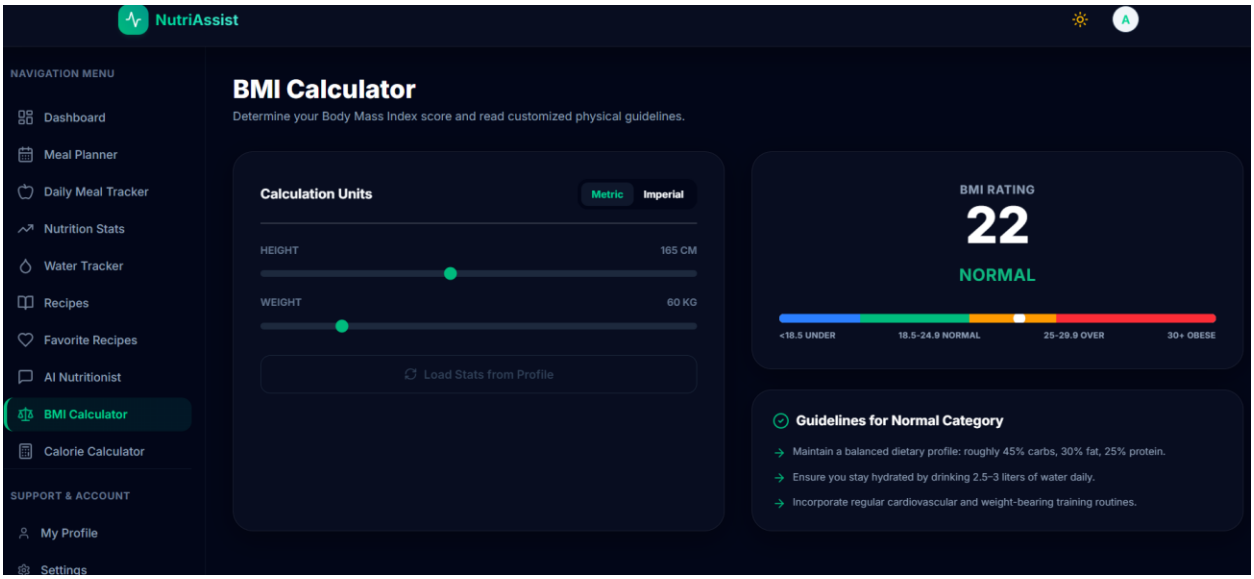


9.3 Dashboard Overview

After authentication, users land on the Dashboard, summarizing today's calories, protein, carbs, fat and water against their daily goals, plus a 7-day trend chart.

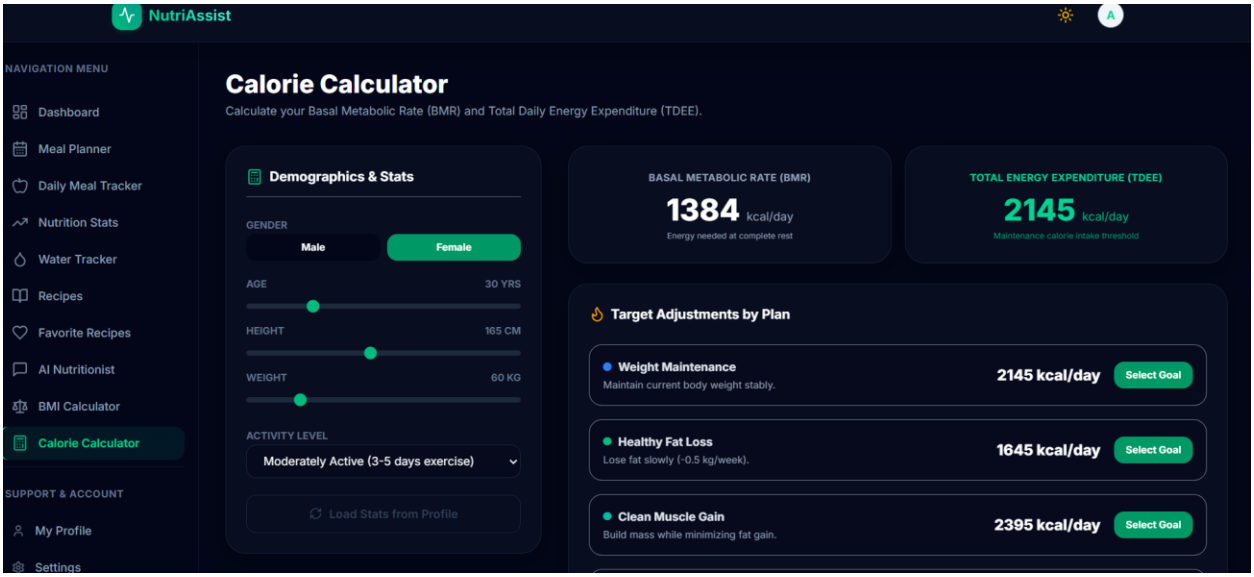
9.4 BMI Calculator

A real-time BMI calculator with a metric/imperial toggle, category banding and tailored guidance tips.



9.5 Calorie Calculator

A BMR/TDEE calculator (Harris-Benedict formula) with activity-level multipliers and goal-based calorie targets that can be applied directly to the profile.

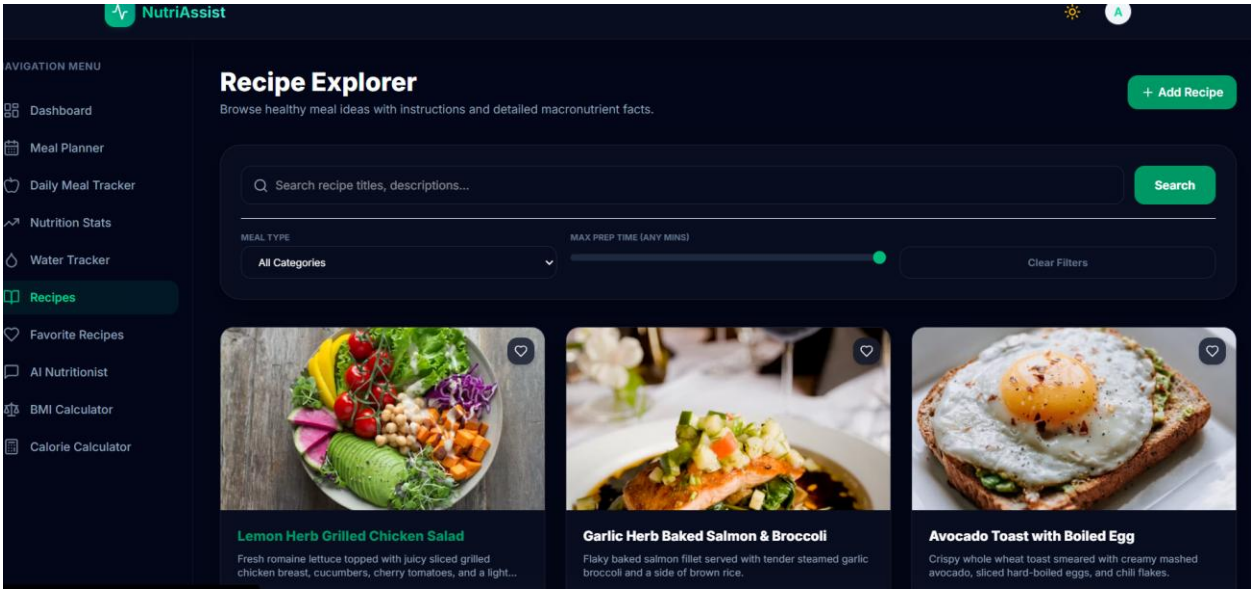


9.6 Daily Tracker

Users log food under a meal slot (breakfast/lunch/dinner/snack) and log water intake, with totals recalculating automatically.

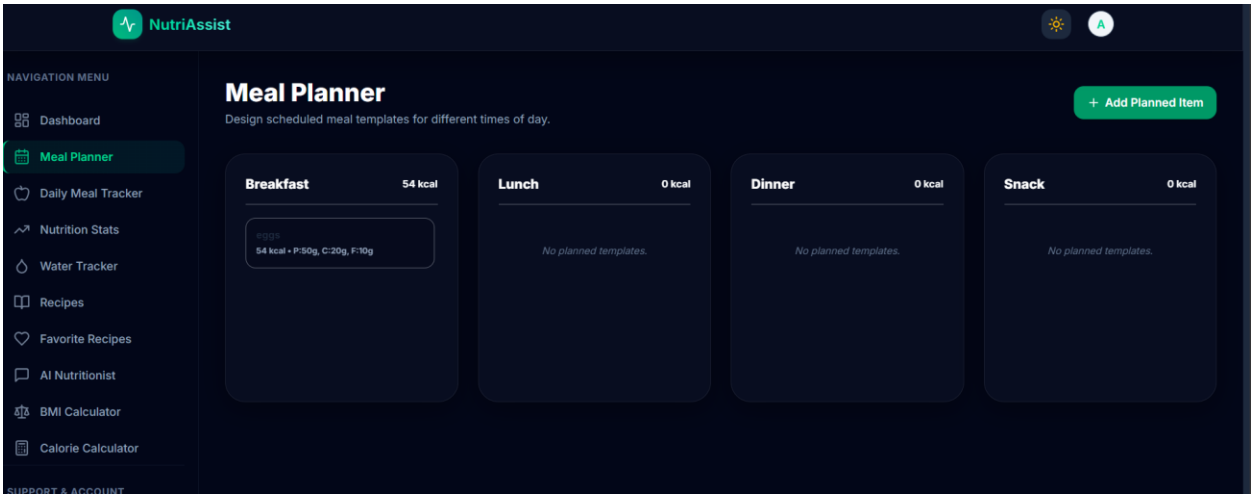
9.7 Recipe Explorer

A searchable, filterable, paginated recipe catalog with cook-time and calorie filters, and a favorite toggle.



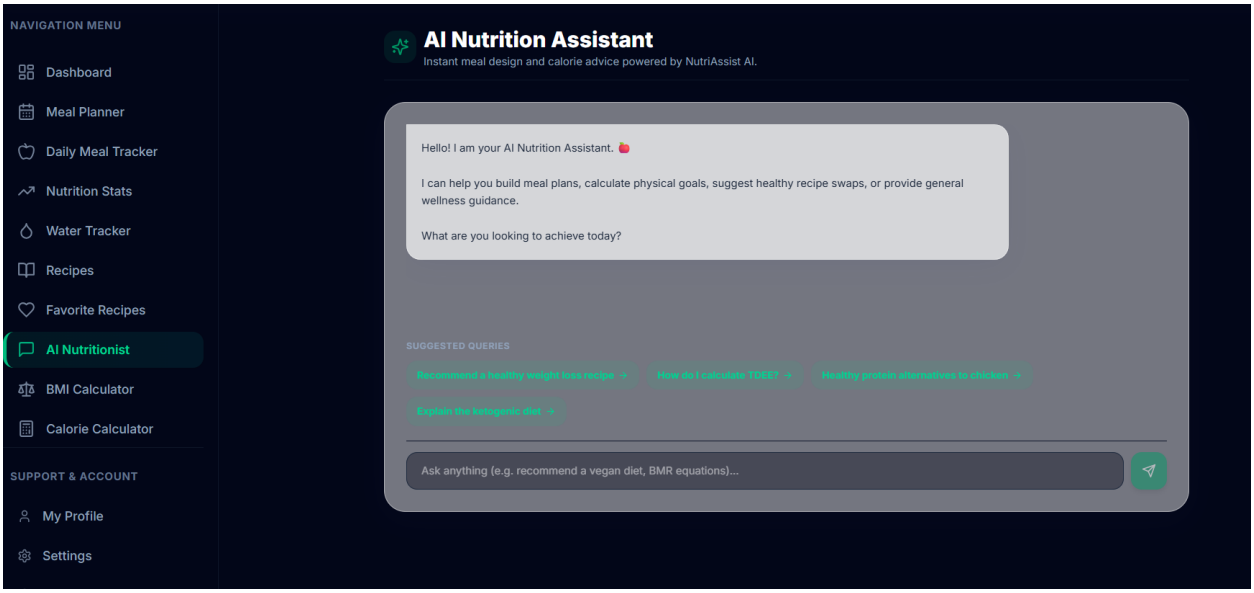
9.8 Meal Planner

Users schedule planned meals by time of day, optionally with an uploaded image.



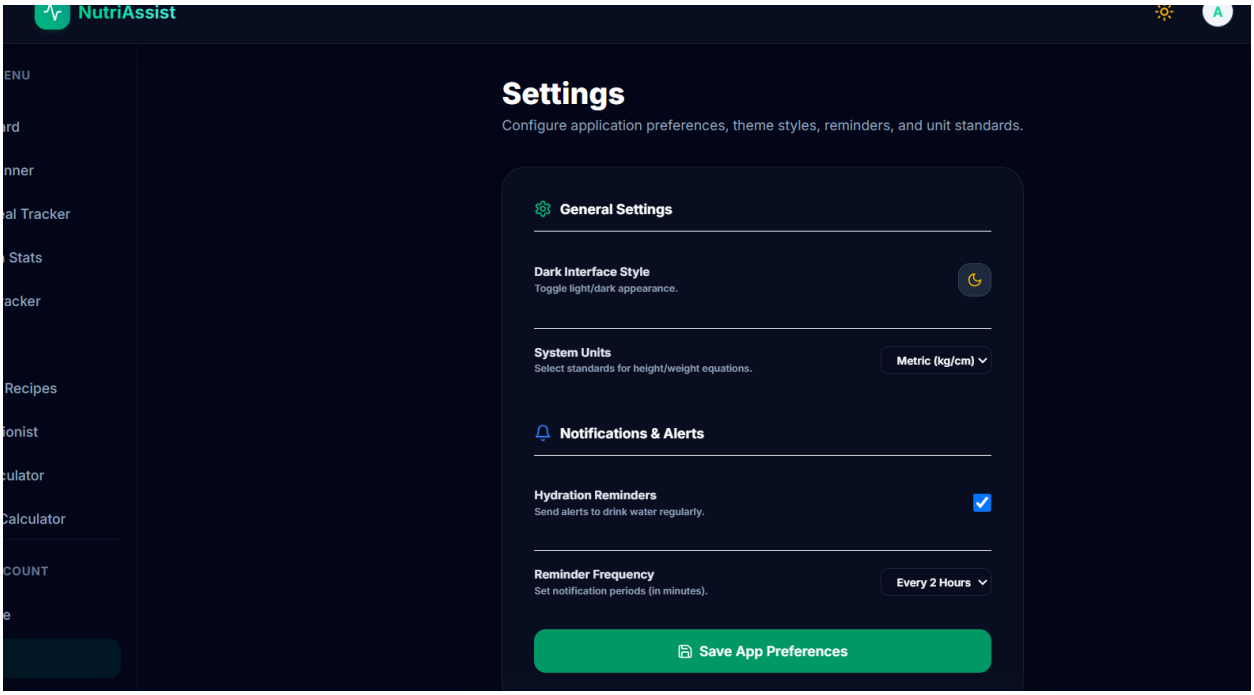
9.9 AI Nutrition Assistant

A chat interface where users ask nutrition questions and receive Gemini-generated (or fallback) guidance formatted in markdown.



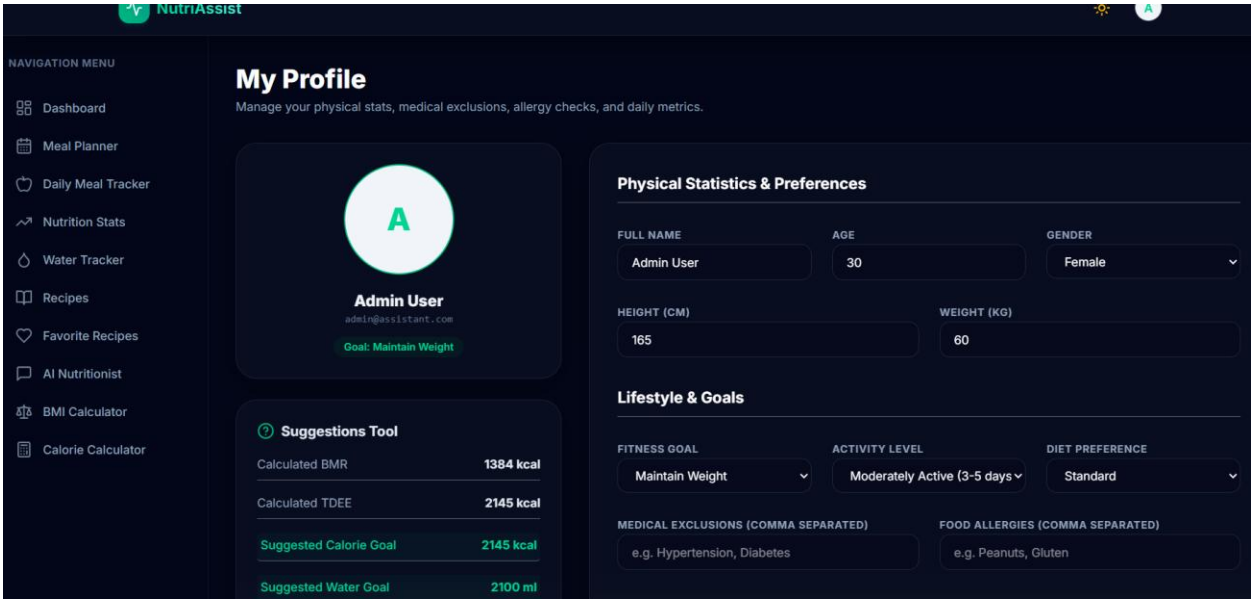
9.10 Settings

Dark mode toggle, unit system (metric/imperial) and configurable water-reminder interval.



9.11 Admin Dashboard

Administrators view platform statistics — total users, recipes, system foods and today's active logs — alongside recent signups.



9.12 Admin — User & Food Management

Administrators manage the user list (with cascade delete) and curate standard system foods.

10. Testing

Nutrition Assistant was validated through a combination of Node.js assert-based unit tests, manual functional testing, and API integration checks.

Test Type	Coverage
Unit Testing (calculations.test.js)	Validated BMI, BMR (male/female) and TDEE formula correctness against known reference values
API Integration Testing (api.test.js)	Verified base route and /api/recipes endpoint respond with the expected status and payload shape
Functional Testing	Validated core user flows: registration, login, calculators, food/water logging, recipe search
Authentication Testing	Verified JWT issuance, refresh rotation, and route protection
Database Testing	Verified Mongoose schema validation, pre-save hook aggregation and unique compound indexes
UI & Responsive Testing	Verified layouts and dark mode across desktop and mobile breakpoints

Sample Test Scenarios

- BMI calculation returns 24.2 for 70kg/170cm and 34.6 for 100kg/170cm
- Male BMR (age 25, 170cm, 70kg) falls between 1600–1800 kcal via the Harris-Benedict formula
- TDEE for a 1500 kcal BMR at "sedentary" activity level equals 1800 kcal
- A NutritionLog’s totalNutrition recalculates correctly whenever a meal is added or removed
- A WaterLog’s amount recalculates correctly whenever an entry is added or removed

11. Screenshots / Demo Walkthrough

A typical end-to-end walkthrough of the platform follows the sequence below, tying together the screens captured in Section 9.

Step	Screen	Description
1	Landing Page	Visitor learns about calculators, tracking and the AI chatbot
2	Register / Login	User creates an account and signs in
3	BMI / Calorie Calculator	User computes BMI and a personalized calorie target
4	Dashboard	User reviews today's progress and weekly trend
5	Daily Tracker	User logs a meal and water intake
6	Recipe Explorer	User searches recipes and favorites one
7	AI Assistant	User asks a nutrition question and gets a response
8	Admin Dashboard	Administrator reviews platform statistics and manages a system food

For a live, interactive demonstration, refer to the deployed application URL or the demo video shared alongside this documentation.

12. Known Issues

- Online payment / premium subscription tier is not yet implemented
- Password reset tokens are stored in-memory on the server and are lost on server restart
- The configurable water-reminder interval in Settings does not yet trigger push/email notifications
- Admin food management supports one-by-one create/update/delete rather than bulk actions
- Cloudinary uploads fall back to local disk storage when cloud credentials are not configured, which is not persistent across some deployment platforms

13. Future Enhancements

- AI-personalized weekly meal plans generated from a user's logged history
- Wearable device integration (steps, heart rate, activity) for automatic calorie-burn tracking
- Push and email notification system for water reminders and daily goal summaries
- Native mobile application (iOS & Android)
- Persistent, database-backed password reset token storage
- Bulk admin actions for food and recipe moderation
- Premium subscription tier with advanced analytics and priority AI chatbot access

Conclusion

Nutrition Assistant is a complete MERN Stack web application that brings BMI/calorie calculation, food and water tracking, recipe discovery, meal planning, and AI-powered nutrition guidance together into a single, cohesive platform. By combining secure authentication, automatic nutrition aggregation, responsive design and an integrated AI chatbot, Nutrition Assistant lays a strong foundation for evolving into a comprehensive, AI-assisted personal wellness ecosystem.